



Crescent Club of Finance

Official Website & Event Registration Platform

Product + Engineering Handbook

Version 0.5 — FINAL — Corrected and frozen requirements and architecture

Prepared for the Crescent Club of Finance IT and leadership teams

Prepared by: Rohith Y - IT Team 02 September 2026

Document Control

Field	Value
Document title	Crescent Club of Finance — Official Website & Event Registration Platform — Product + Engineering Handbook
Version	0.5
Status	FINAL — v0.5 requirements and architecture have been finalized for implementation; remaining PENDING items are CCF operational/content decisions.
Primary audience	CCF leadership, authorized CCF representative, IT/development team, future handover teams, and Antigravity implementation agents
Current implementation state	No implementation code is authorized by this document. The handbook is the source of truth to be approved before coding begins.
Source rule	Confirmed facts are preserved from supplied CCF materials. Missing facts are marked pending; recommendations and future scope are explicitly labeled.

Important status note. This handbook consolidates the supplied master prompt, the supplied CCF events document, the supplied logo and website references, and all confirmed revision decisions. The v0.5 requirements and architecture are finalized for implementation. The platform uses a confirmed stack (Next.js, Prisma, PostgreSQL, Auth.js, Cloudflare R2, Resend). Remaining PENDING items are genuine CCF operational or content decisions. Implementation may begin after the approved pending operational decisions are resolved where they affect launch. Per-event content, field choices, capacity, and rules remain marked for confirmation where applicable.

Contents

1	Executive Summary	1
2	Project Vision	1
3	Goals and Success Measures	1
4	Scope and Non-Goals	2
5	Stakeholders and Responsibilities	3
6	Source-of-Truth and Content Governance	3
7	Confirmed CCF Information	4
8	Pending Requirements and Decision Register	5
9	Information Architecture and Sitemap	7
10	Page Specifications	7
	10.1 Home	7
	10.2 About	8
	10.3 Departments	8
	10.4 Members	8
	10.5 Events and event details	8
	10.6 Registration	8
	10.7 Join Us	9
	10.8 Contact	9
	10.9 Admin	9
11	User Journeys	10
	11.1 Visitor to event registration	10
	11.2 Event registration configuration model	10
	11.3 Team registration	10
	11.4 Recruitment	11
	11.5 Export and cleanup	11
12	Design System	11
	12.1 Typography	12
	12.2 Layout and motion	12
	12.3 CCF-specific interaction ideas	12
13	Responsive Design	12
14	Accessibility	12
15	SEO and Performance	13
16	Event System	13
17	Registration System	15
	17.1 Validation order	16
	17.2 Concurrency strategy	16
	17.3 Capacity changes and reopening	17
	17.4 Registration Mode and External Forms	17
	17.5 Internal CCF Event Form Builder	17
	17.6 Conditional Logic for Mixed Events	18

17.7 Rich-Text Event Content	18
17.8 Payment Architecture	18
18 Team Registration	19
19 Recruitment System	20
20 Admin Dashboard	20
21 Notification System	21
22 Media and Event Gallery	21
23 CSV Export, Backup, and Data Lifecycle	21
24 Database Architecture	22
24.1 Constraints and indexes	25
25 API Architecture	25
25.1 Error contract	27
26 Authentication and Authorization	27
27 Security and Privacy	28
28 Infrastructure and Repository Architecture	28
29 Development Workflow and Team Division	29
30 Environment Variables and Account Controls	29
31 Error Handling and Operational States	30
32 Testing Strategy	31
33 Acceptance Criteria	33
34 MVP and Future Scope	34
35 Requirements Traceability Matrix	34
36 Development Roadmap	36
36.1 Phase 0 – Requirements and Architecture	36
36.2 Phase 1 – Repository and Project Setup	36
36.3 Phase 2 – Database Setup	36
36.4 Phase 3 – Authentication	36
36.5 Phase 4 – Design System and Layout	36
36.6 Phase 5 – Public Website	36
36.7 Phase 6 – Event Management	36
36.8 Phase 7 – Dynamic Form Engine	36
36.9 Phase 8 – Registration Engine	36
36.10 Phase 9 – Team Registration	36
36.11 Phase 10 – Payment	36
36.12 Phase 11 – Recruitment	37
36.13 Phase 12 – Admin Dashboard	37
36.14 Phase 13 – CSV and Media	37
36.15 Phase 14 – Security and Testing	37
36.16 Phase 15 – Deployment	37
36.17 Phase 16 – Handover	37
37 Deployment, Backup, and Recovery	37
38 Antigravity Implementation Brief	37

39 Handover Checklist 39

40 Final Quality Audit 39

v0.5 Change Log 40

 v0.5 Final Correction Pass 41

41 References and Source Materials 41

1 Executive Summary

The Crescent Club of Finance (CCF) requires a production-grade official website and digital platform rather than a simple static college-club page. The platform must communicate the club's finance, financial-literacy, leadership, professional-development, industry-exposure, networking, student-activity, event, practical-learning, and club-culture identity while providing dependable operational workflows for events, registrations, recruitment, media, administration, export, and handover.

The implementation technology stack is confirmed. The platform will be built as a Next.js full-stack application using TypeScript, React, Tailwind CSS, shadcn/ui, and Motion for the frontend. The backend uses Next.js Route Handlers with Zod validation. Data is managed in a PostgreSQL database using the Prisma ORM. Admin authentication is provided by Auth.js using passwordless magic links via Resend, with TOTP fallback and one-time recovery codes. Media storage is finalized as Cloudflare R2. Deployment is on Vercel, with Sentry for monitoring. This stack is the implementation baseline.

The most important engineering requirement is correctness under registration load. The system must treat each CCF event as a configurable product that independently defines who may register, how they register, what information is collected, whether teams are allowed, whether payment is required, how capacity is enforced, and what content/rules are displayed. The system must validate Crescent RRN format only when required for a Crescent participant, identify external participants using normalized college plus normalized external roll number, prevent duplicate active participation at the event scope, and atomically reserve capacity so that simultaneous registration or payment attempts cannot exceed the event limit. Authorized deletion must release capacity and allow re-registration. The platform must not retain unnecessary student data permanently.

The most important governance requirement is source integrity. The implementation team must preserve official CCF terminology and must not invent members, positions, faculty details, event claims, statistics, registration fields, or hierarchy. Any unresolved issue must appear in the pending-decisions register and be approved by CCF before it is treated as a confirmed requirement.

2 Project Vision

> Premium finance-club identity with modern digital product quality.

The public experience should feel premium, modern, intelligent, professional, youthful, finance-oriented, elegant, and technically polished. It must not resemble a generic college template, a corporate banking website, an overloaded stock-market dashboard, or a flashy animation showcase.

The existing CCF logo is the primary visual source of truth. The design direction should therefore derive from its dark navy or black, gold or amber, and white or off-white visual language. Typography, spacing, component rules, and animation may be established for the website, but they must remain consistent with the club identity.

3 Goals and Success Measures

The platform's goals are to establish a credible official digital presence, make verified club information discoverable, reduce manual event-registration work, support both individual and team events, provide a controlled recruitment intake, give authorized users reliable data exports, organize event media, and leave future IT teams with a comprehensible system.

Goal	Launch success condition
Public identity	A visitor can understand what CCF is, what it does, and how to contact or join it without relying on internal knowledge.
Event operations	An authorized IT user can configure, publish, close, archive, and export an event without code changes for ordinary event variations.
Registration integrity	Invalid Crescent RRN formats, external identity collisions, duplicate active participation, invalid team configurations, late submissions, payment-state errors, and capacity race conditions are rejected safely.
Recruitment	CCF can manually open and close recruitment, view/search/export applications, and expose the WhatsApp action only when configured.
Privacy	Public pages never expose RRNs, private phone numbers, internal identifiers, registration records, recruitment records, or administrative APIs.
Handover	A future IT team can deploy, operate, test, back up, and update the platform from the handbook and repository documentation.

4 Scope and Non-Goals

The MVP includes the public website, About, Departments, Members, Upcoming Events, Previous Events, event-detail pages, individual registration, configurable team registration, recruitment, optional WhatsApp redirects, an authenticated admin dashboard, CSV export, event media/gallery, authentication, responsive design, security controls, SEO, performance work, configurable form versions, and internal manual UPI payment verification for paid events.

The MVP does not include certificates, attendance tracking, student accounts, automated certificate generation, advanced analytics, event check-in QR codes, alumni management, sponsor management, newsletters, or an advanced CMS unless CCF explicitly adds them and updates the approved scope. These are **[FUTURE SCOPE]**. Payment configuration and payment-state architecture are part of the MVP because some events are free and some are paid; the provider and final payment/refund policy remain pending.

The website should not require public student accounts. Event registration may use the internal CCF form builder, an approved external form/link, or no online registration according to the event's Registration Mode. Crescent students use Crescent RRN where configured: normalize first, then require exactly 12 digits beginning with 2, digits only. External students use Name, College / University name, External Roll Number, and other configured fields; they must not be forced to provide Crescent RRN. External uniqueness is event-scoped at EVENT + normalized college + normalized external roll number, so the same roll number may be used by students from different colleges and in different events. CCF/IT currently has

no college student database or RRN verification API, so institutional verification is **[FUTURE SCOPE]**. The administrative area must require authentication.

5 Stakeholders and Responsibilities

CCF leadership owns the official content, confirms pending decisions, approves public-facing claims, and authorizes access to student data. The authorized CCF representative is an operational administrator. The three-person IT/development team owns technical setup, implementation, deployment, maintenance, event configuration, exports, backups, and handover. Future batches inherit the repository, deployment accounts, database, storage, documentation, and recovery procedures.

Stakeholder	Responsibilities	Approval boundary
CCF leadership	Official facts, public claims, recruitment policy, event rules, faculty/coordinator details	Approves content and policy
Authorized CCF representative	Operational review, event requirements, registration exports, data handover	Authorized administrative access
IT/development team	Architecture, code, environments, event setup, security, exports, backups, deployment	Technical control; must follow handbook
Future IT team	Maintenance, updates, credential rotation, incident response, content operations	Receives documented handover
Public visitor/participant	Browse content, discover events, and submit a permitted internal registration, external-form registration, or recruitment application	No administrative access

6 Source-of-Truth and Content Governance

The supplied master prompt establishes the source-of-truth rule: when information is explicitly present in CCF materials, preserve its terminology, names, structure, descriptions, and organizational information. The implementation must not silently invent information. Any missing item must be written as **[PENDING CCF CONFIRMATION]**. Any design or technical choice that is not a CCF requirement must be written as **[ARCHITECTURAL RECOMMENDATION]** or **[ARCHITECTURAL/DESIGN RECOMMENDATION]**. Future enhancements must be marked **[FUTURE SCOPE]**. Conflicting sources must be written as **[CONFLICT REQUIRES CCF CONFIRMATION]**.

Label	Meaning	Required action
Confirmed	Explicitly provided by CCF source material or the approved requirements	May be implemented as stated

Label	Meaning	Required action
PENDING CCF CONFIRMATION	Required information is missing or ambiguous	Do not hardcode as an official fact
ARCHITECTURAL RECOMMENDATION	A technical choice proposed for maintainability or security	CCF may approve or replace it
ARCHITECTURAL/DESIGN RECOMMENDATION	A visual or interaction choice not defined by CCF	Keep consistent with logo and accessibility
FUTURE SCOPE	Not required for MVP	Do not block launch unless later approved
CONFLICT REQUIRES CCF CONFIRMATION	Two supplied sources disagree	Preserve both in the issue register; do not silently choose one

7 Confirmed CCF Information

The supplied materials identify the organization as **Crescent Club of Finance** at B.S. Abdur Rahman Crescent Institute, Vandalur, Tamil Nadu, India. The official email is crescentcluboffinance26@gmail.com. The supplied official social links are Instagram at <https://www.instagram.com/crescentcluboffinance/> and LinkedIn at <https://www.linkedin.com/company/ccf-2024/>.

The supplied CCF events document describes the club as a student-driven initiative fostering financial literacy, interdisciplinary collaboration, and professional growth. Its stated vision is to create a financially literate community in which students from all backgrounds gain confidence to make informed financial decisions for life. Its stated mission is to equip students, including non-finance majors, with essential financial knowledge and money-management skills through interactive learning and practical experiences. The supplied objectives are holistic skill development through workshops and simulations, interdisciplinary collaboration through projects integrating diverse academic perspectives, and a lifelong financial-learning hub that creates societal impact through financial-literacy outreach. The supplied leadership structure describes a President, Vice President, Managing Director, directors and joint directors for Events, Finance, Marketing, IT/Media, and Project Management, together with Advisors.

The supplied events document lists FinVibe Fiesta as an intra-college event in February 2024, Tax Savvy Strategy as a faculty tax seminar in March 2024, Millionaire Mindset as a finance-wellness seminar in April 2024, FinCres24 Summit as a national-level summit in August 2024, and FinVibe Fiesta Season 02 as an international event in April 2025. The master prompt additionally identifies Magnora'26, FinRise'25, and FinVibe Fiesta S2 as initial events to support. CCF has confirmed that event participants may be Crescent students or students from other colleges/universities; participants may be UG or PG, from any academic department and any year of study. Mixed Crescent + external events are a first-class supported configuration, and mixed teams are allowed. External students are identified by Name, College / University name, External Roll Number, and other configured fields. The expected peak registration volume for the

largest event is approximately 500 students, separate from each event's configured capacity. Recruitment is currently open, while the system must still support manual opening and closing. CCF will provide complete event-specific content and rules when each event is handed over to IT; those per-event values must not be invented in advance.

The confirmed admin board is Remi Kayalvizhi — President; Fizza Fathima — Vice President; and Zayan Ahmed — Managing Director. The supplied PDF also names Dr. Karthikeyan, Dean — Student Affairs, and Dr. K. Indra Gandhi, Deputy Dean — Student Affairs, as club coordinators. These confirmed leadership names and positions must be used consistently in approved public leadership content.

8 Pending Requirements and Decision Register

The following items must remain visible until a named owner confirms them. The owner column is a working recommendation and should be replaced with the actual responsible person or body.

Decision	Status	Owner	Impact
Expected peak registration volume	CONFIRMED: approximately 500 participants	CCF + IT	Use approximately 500 participants as the platform/load-testing planning target; do not confuse this with an event's configured capacity.
Capacity mode per event	Confirmed configurable rule; event value pending	CCF + IT	Every team event must explicitly set PARTICIPANTS or TEAMS; no event value may be guessed.
Exact fields and rules by event	PENDING PER EVENT	CCF event owner + IT	CCF supplies the actual fields, eligibility details, rules, capacity, dates, fees, and team requirements for each event; the platform architecture is confirmed as configurable.
Previous-event display	PENDING	CCF	Determines public event cards, detail pages, galleries, and historical claims.
Member eligibility and profile rules	PENDING	CCF	Controls public member directory and department mapping.
Faculty/coordinator confirmation	PENDING CCF CONFIRMATION	CCF	Prevents publication of incorrect institutional information; this remains separate from the now-confirmed admin-board names.

Decision	Status	Owner	Impact
Data retention duration	PENDING	CCF + IT	Controls export, archive, deletion, backup, and privacy operations.
Final domain	PENDING	CCF	Controls canonical URLs, email links, SEO, and ownership.
Infrastructure stack	CONFIRMED: Next.js, Prisma, PostgreSQL, Auth.js, Resend, Cloudflare R2, Vercel, Sentry	IT	Finalized implementation stack for operations and handover.
Media storage provider	CONFIRMED: Cloudflare R2	IT	Finalized production object-storage backend.
Notification channels	PENDING	CCF + IT	Dashboard notifications are recommended; email is optional; Telegram is future scope.
Technical account ownership	PENDING	CCF + IT	Controls continuity across student batches and incident recovery.
External participant identity	CONFIRMED: Name + College / University name + External Roll Number; validation details per event	CCF + IT	External uniqueness is EVENT + normalized college + normalized external roll number; no global college or roll-number uniqueness.
Payment provider	PAYMENT_PROVIDER optional future capability; MANUAL_UPI is confirmed MVP method	CCF + IT	Do not make Razorpay or another gateway mandatory; provider integration may be added later.
Payment/refund policy	PENDING CCF CONFIRMATION	CCF	Define refunds, expiry, failure handling, and operational treatment of paid INTERNAL registrations; EXTERNAL payment remains outside the CCF internal payment database.

Decision	Status	Owner	Impact
Conflicting source facts	PENDING CONFIRMATION	CCF	Must be resolved before public publication.

9 Information Architecture and Sitemap

The public navigation is HOME, ABOUT, DEPARTMENTS, MEMBERS, EVENTS, CONTACT, and JOIN US. DEPARTMENTS is required in the primary navigation. JOIN US represents the current recruitment process.

CRESCENT CLUB OF FINANCE

- └─ Home /
- └─ About /about
- └─ Departments /departments
 - | └─ Department detail /departments/:slug
- └─ Members /members
- └─ Events /events
 - | └─ Upcoming events /events?status=upcoming
 - | └─ Previous events /events?status=previous
 - | └─ Event details /events/:slug
 - | └─ Registration /events/:slug/register
- └─ Contact /contact
- └─ Join Us /join-us
- └─ Admin /admin
 - └─ Dashboard
 - └─ Events
 - └─ Registrations
 - └─ Recruitment
 - └─ Members
 - └─ Departments
 - └─ Media / Gallery
 - └─ Notifications
 - └─ Settings

All public pages require loading, empty, error, responsive, accessibility, and SEO behavior. Admin pages prioritize speed, tables, search, filters, export, status indicators, and confirmation dialogs rather than decorative complexity.

10 Page Specifications

10.1 Home

The Home page establishes CCF identity and routes visitors to events, departments, members, and recruitment. It should contain the hero, verified club introduction, vision and mission, department preview,

upcoming events, previous-event preview, leadership preview, a content-based reason-to-join section, primary calls to action, and the footer. Statistics may appear only when verified; otherwise the page must use qualitative content.

The hero should use the supplied logo and a restrained finance-inspired visual system: grid lines, gold accent rules, subtle market or timeline motifs, and editorial typography. It must not become a trading dashboard. On mobile, the hero becomes a stacked layout with readable text and an immediate event or Join Us action.

10.2 About

The About page presents the official overview, history or story when confirmed, vision, mission, objectives, activities, and reason for existence. Official wording should be preserved in meaning. Marketing rewrites must be labeled as design or editorial recommendations. The page should include a CTA to Events and Join Us.

10.3 Departments

The directory uses the supplied names: Finance Management, IT & Media, Marketing & PR, Project Department, and Event Management. Each department page or section may show an approved description, icon, approved leadership, and public members. RRN, phone numbers, student identifiers, and private academic information are prohibited.

10.4 Members

The Members page shows public profiles containing only approved name, position, department, profile photo, short description, and approved social link. The exact hierarchy must come from confirmed CCF material. The directory must be data-driven so that authorized IT users can update records without rewriting the site.

10.5 Events and event details

The Events page separates Upcoming Events and Previous Events, with status labels REGISTRATION NOT OPEN, REGISTRATION OPEN, REGISTRATION CLOSED, and EVENT FULL. Event detail pages show the poster or banner, verified description, date, time, venue, category, eligibility, rules, speaker or guest information where confirmed, registration mode, registration fee where applicable, registration status, capacity indication where appropriate, the configured capacity mode where useful, and an optional WhatsApp action only when the event has a configured link.

10.6 Registration

The registration page is event-configured. For Registration Mode = INTERNAL, it renders the internal CCF Event Form Builder output: participant type, conditional Crescent or external path, system fields, custom fields, individual or team mode, team-name and team-leader controls, member rows, participant-specific identifiers, phone/email fields, payment steps where required, the configured minimum and maximum team size, and the event's explicit capacity mode: PARTICIPANTS or TEAMS. For Registration Mode = EXTERNAL, it shows the configured external CTA instead of duplicating the external form. For Registration Mode = NONE, it shows no registration CTA. On internal success it displays a success state and, only when configured, a "Join WhatsApp Group" action. Students cannot edit or cancel through the public interface.

10.7 Join Us

The Join Us page reflects the manually controlled recruitment state. While open, it presents the confirmed eligibility and fields: name, RRN, department applying for, academic department, year of study, and WhatsApp-enabled phone number. The recruitment RRN field uses the same server-side format rule: exactly 12 digits, beginning with 2, digits only after normalization. It does not require a resume, questionnaire, or interview. After successful submission it shows the configured recruitment WhatsApp link. When closed, it displays “Recruitment Closed,” removes or disables Apply Now, and rejects submissions server-side. Recruitment remains a separate CCF process and is not changed by the new event-registration eligibility rules. External-college students may register for eligible events but may not apply for CCF recruitment merely because event participation is open to them.

10.8 Contact

The Contact page shows the official email, location, Instagram, LinkedIn, and only confirmed relevant club or faculty information. It must not include a general WhatsApp group link.

10.9 Admin

The Admin area is private. It contains Dashboard, Events, Registrations, Recruitment, Members, Departments, Media/Gallery, Notifications, and Settings. Destructive actions require confirmation, and raw database or server errors are never shown to users.

Route	Primary data	Empty state	Error/accessibility requirement
Home	Site settings, events, departments, approved members	Use verified editorial blocks; never invent statistics	Semantic sections, keyboard CTA access, descriptive image alt text
Events	Published events and status	“No events are currently published”	Status text must not rely on color alone
Event detail	Event, fields, media, registration state	“Event details are being prepared”	Closed/full states remain actionable and understandable
Registration	Event fields and configuration	Registration disabled state	Server errors mapped to human-readable messages
Members	Approved member profiles	“Member profiles will be added soon”	Keyboard-accessible cards and meaningful alt text
Join Us	Recruitment settings and fields	Recruitment Closed	Labels, validation, and focus recovery are required
Admin tables	Protected records	No records found	Search/filter controls must be keyboard and screen-reader accessible

11 User Journeys

11.1 Visitor to event registration

Visitor → Events → Event detail → Read eligibility/mode/status → INTERNAL form OR EXTERNAL CTA OR no CTA → Conditional fields/payment if internal → Server validation → Atomic transaction or trusted payment confirmation → Success → Optional WhatsApp action

The event detail page is the decision point. The event configuration determines eligibility, registration mode, form content, payment behavior, capacity semantics, and CTA. The server must re-check every important condition even if the browser has already validated it. External mode opens the configured external URL and does not import or replicate responses.

11.2 Event registration configuration model

CCF events are configurable products. These concepts are deliberately separate:

Concept	Question answered	Configuration
Event eligibility	Who can participate?	Crescent Students ENABLED/DISABLED; Other Colleges ENABLED/DISABLED. Valid states include Crescent-only, external-only, and mixed.
Registration Mode	How do they register?	INTERNAL, EXTERNAL, or NONE
Form configuration	What information is collected?	System fields, custom fields, field order, validation, and conditional logic for internal events
Participation model	Individual or team?	Team leader/name requirements, min/max size, member identity rules, mixed-team policy
Payment configuration	Is a fee required?	FREE or PAID, amount, payment status behavior, provider boundary
Capacity configuration	How is capacity enforced?	PARTICIPANTS, TEAMS, or UNLIMITED, with concurrency-safe atomic reservation
Content configuration	What is displayed?	Rich-text description, rules, instructions, eligibility explanation, notes, links

Every event must be configured through the Admin Panel using these independent dimensions. The actual per-event values are supplied by CCF with the event brief and must not be invented.

11.3 Team registration

Participant → Team mode → Team details → Optional leader/name → Member rows → Crescent/external participant path per configuration → RRN/identifier rules →

Event-level duplicate check → Size/capacity-mode transaction → Optional payment → Success

A team must not require a leader relationship unless the event configuration says so. Team membership is a relationship, not a fixed hierarchy. Mixed teams may include Crescent and external participants only when the event configuration explicitly allows them; mixed-team eligibility is not assumed universally.

11.4 Recruitment

Student → Join Us → Recruitment-open check → Application form → RRN and field validation → One-department uniqueness check → Save → Success → Optional configured WhatsApp group

Recruitment closure is manual. A calendar date must not close recruitment automatically unless CCF later changes the policy.

11.5 Export and cleanup

Event completed → Authorized admin starts export → Generate dynamic CSV → Download/checksum or row-count verification → Share/archive → Confirm handover → Delete registration records

The deletion action is an authorized admin operation and must require confirmation with the deletion scope stated. For completed-event cleanup, it follows successful export verification and handover. For an active/current registration deletion permitted by policy, its capacity allocation is released and the participant may register again for the same event; the duplicate guard applies to active/current participation only. The system must retain a non-sensitive audit record of who performed the export, deletion, capacity release, and any re-registration-related lifecycle action without retaining unnecessary student fields.

12 Design System

The design system is a [ARCHITECTURAL/DESIGN RECOMMENDATION] derived from the supplied logo. It uses a dark premium foundation, gold or amber accents, off-white typography, restrained borders, high contrast, and editorial display treatment combined with a readable sans-serif UI face.

Token family	Recommended token	Usage
Navy foundation	#071426 / #0B1D33	Shell, navigation, hero overlays, admin chrome
Gold accent	#D7A63D / #C6902D	Primary CTA, active navigation, rules, status emphasis
Off-white	#F7F4EC	Main text on dark surfaces and editorial headings
Muted text	#A8B0BD	Supporting copy and metadata
Surface	#101F34	Cards, tables, panels, form surfaces
Success	#4FA77A	Accessible success state with text label

Token family	Recommended token	Usage
Warning	#D6A24A	Capacity or closing-soon state with text label
Error	#D16A63	Validation and destructive-action state with text label

12.1 Typography

Use an elegant serif or editorial display face for major titles and a highly readable sans-serif for navigation, forms, tables, and admin surfaces. This is **[ARCHITECTURAL/DESIGN RECOMMENDATION]** because CCF has no official typography specification. The implementation should verify actual font availability and provide fallbacks. Recommended hierarchy: display 3.5–5rem on large screens, 2.4–3.1rem on mobile; body 1rem with 1.55–1.7 line-height; labels 0.82–0.92rem; metadata 0.78–0.88rem.

12.2 Layout and motion

Use a 4px or 8px spacing base, 12–18px card radius, subtle borders, and soft shadows rather than glass-heavy effects. Animation should be purposeful and short: scroll reveals, hover states, page transitions, line or timeline motion, and button feedback. Avoid constant movement, excessive parallax, particles, huge transitions, and unnecessary animation libraries. Respect `prefers-reduced-motion` by replacing transitions with immediate state changes or a short opacity change.

12.3 CCF-specific interaction ideas

The MVP should include only features that reinforce information hierarchy. Recommended MVP ideas are event cards with a market-style status ribbon, a finance-inspired event timeline, a compact capacity indicator, department “sector” cards, a restrained gold line motif, and a data-like activity label for event dates. A “Crescent Index” or finance ticker can be explored later only if it communicates real verified information. A finance-themed cursor, animated statistics, and portfolio-style member layout are **[FUTURE SCOPE]** unless approved after usability review.

13 Responsive Design

The responsive system must be designed for mobile, tablet, laptop, desktop, and large desktop rather than treating mobile as a compressed desktop. The recommended breakpoints are **[ARCHITECTURAL RECOMMENDATION]**: 0–639px mobile, 640–1023px tablet, 1024–1279px laptop, 1280–1535px desktop, and 1536px and above large desktop.

On mobile, navigation becomes a drawer or compact menu, event cards become a single column, registration fields become stacked controls, team members become repeatable full-width rows, gallery images use a one- or two-column layout, and admin tables become horizontally scrollable or transform into labeled cards. The registration form must preserve large touch targets, visible labels, field-level errors, and a persistent summary of the event and team configuration.

14 Accessibility

The platform must use semantic HTML, logical heading order, keyboard navigation, visible focus states, labels associated with inputs, accessible buttons, screen-reader-friendly navigation, alt text for meaningful

images, and sufficient contrast. ARIA should be used only when native HTML does not express the behavior. Errors must be written in plain language and announced or associated with the affected field. Color must never be the sole status indicator.

Registration forms require explicit labels, examples for RRNs and phone numbers where helpful, a summary of errors at the top, field-level messages, preserved user input after recoverable errors, and focus movement to the first invalid field. Reduced-motion behavior is mandatory.

15 SEO and Performance

Public pages should have unique title and description metadata, canonical URLs, Open Graph metadata, appropriate Twitter/X metadata, a generated sitemap, robots.txt, and structured data only where the underlying facts are confirmed. Individual event pages may use Event schema only when the event name, date, location, and status are verified. No fake statistics or unsupported SEO claims may be introduced.

CCF has confirmed an expected peak registration volume of approximately 500 students for the largest event. This is a platform/load-testing planning target, not a default event capacity; each event's actual capacity remains independently configurable. Performance targets are **[ARCHITECTURAL RECOMMENDATION]**: fast first load on ordinary Indian mobile networks, optimized WebP/AVIF media, lazy loading below the fold, responsive image sizes, server-rendered or statically generated public content where practical, code splitting for the admin area, CDN delivery, efficient indexed queries, and limited client-side JavaScript. The launch quality gate should test approximately 500-student load, concurrent registration, and concurrent paid-registration behavior, in addition to Core Web Vitals and basic throttled mobile behavior.

16 Event System

Events are a core system. Every event is a configurable product with public content and an operational configuration. The event lifecycle is Draft → Published → Registration Not Open → Registration Open → Registration Closed or Event Full → Completed → Archived. Registration state is derived from publication, Registration Mode, opening and closing timestamps, eligibility, payment configuration, capacity, and manual closure. CCF has defined the configurable event system; what remains pending is the actual content and rules of each individual event.

Configuration group	Fields	Notes
Identity	Event ID, name, slug, poster, banner	Slug is public and immutable after publication unless redirected
Content	Description, rules, instructions, eligibility explanation, notes, links	Rich text is sanitized before public rendering
Schedule	Date, start time, end time, venue, timezone	Use Asia/Kolkata unless CCF confirms another timezone
Classification	Category/type, speaker/guest	Only confirmed public claims are rendered

Configuration group	Fields	Notes
Eligibility	Crescent Students ENABLED/ DISABLED, Other Colleges ENABLED/ DISABLED	Independent per-event flags; Crescent-only, external-only, and mixed are valid
Registration	Registration Mode INTERNAL/ EXTERNAL/NONE, Registration Method BUILT_IN/GOOGLE_FORM/ NONE, opening/closing time, capacity, capacityMode, registration_opens_at, registration_closes_at, configured fields, active form version	Path, method, window, version, and fields are explicit per event; event details are supplied by CCF
External registration	External URL, button label, short instruction	Required and validated before EXTERNAL publication; no response import; payment remains external
Internal form	System fields, custom fields, order, validation, conditional logic	Focused CCF builder; not a full Google Forms clone
Team rules	Individual/team, leader required, name required, min/max size, participant identity rules, mixed-team policy, capacityMode	No forced leader relationship; every team event chooses PARTICIPANTS or TEAMS
Payment	FREE/PAID, MANUAL_UPI/ PROVIDER, amount, UPI ID, payee name, reference requirement, verification state	MANUAL_UPI is confirmed MVP; provider is optional future capability; never trust client- only success
Communication	WhatsApp enabled, group URL	Show action only when configured
Lifecycle	Draft, published, not open, open, closed, full, completed, archived	Admin transitions are authorized and auditable

The event detail page should explain status clearly. REGISTRATION NOT OPEN means the event is published but the configured opening time has not arrived. REGISTRATION OPEN means a valid submission may be accepted through the configured mode. REGISTRATION CLOSED means the configured closing time passed or an administrator closed the event. EVENT FULL means the configured capacity rule has been reached. For EXTERNAL events, publication requires a valid external URL. For NONE events, no registration CTA is shown. A closed or full event must not display an active internal form. The approximately 500-student expected peak load is a platform planning target and must not be displayed as the event's capacity unless CCF configures that value for the event.

17 Registration System

The public registration process requires no student account. It uses event configuration, eligibility routing, Registration Mode, internal form schema where applicable, event-level duplicate detection, payment state where applicable, and server-side authorization of the current event state. Crescent participants use Crescent RRN only where configured; the normalized value must contain exactly 12 digits beginning with 2 and digits only. External participants use Name, College / University name, External Roll Number, and other configured fields, and must never be forced to provide Crescent RRN. External identity is event-scoped at EVENT + normalized college + normalized external roll number. Uniqueness is enforced through the event_participants identity registry. A participant cannot edit or cancel after internal submission. An authorized deletion releases its active capacity allocation and the associated event_participant identity locks, allowing the participant to register again for the same event. The default internal success path is submission → validation → payment coordination where applicable → database transaction → success state → optional WhatsApp action.

Rule	Required behavior
Eligibility	Route only allowed participant types. Crescent and Other Colleges flags are independent; mixed events are first-class.
Registration Mode	INTERNAL uses the CCF builder; EXTERNAL uses a configured URL; NONE has no CTA.
Crescent RRN	Required only when configured for a Crescent participant; normalize and validate exactly 12 digits beginning with 2, digits only.
External identity	Require Name, College / University name, External Roll Number, and configured fields; normalize college and roll number.
Unique participation	Crescent: EVENT + normalized RRN. External: EVENT + normalized college + normalized external roll number. Enforced via event_participants registry for active participation only. Different colleges may share a roll number; different events may share the same identity.
Registration window	Reject before opening and after closing, regardless of browser state.
Capacity	PARTICIPANTS, TEAMS, or UNLIMITED mode. Reject when full using an atomic transaction so concurrent requests/payments cannot overbook.
No waiting list	Do not create a waiting-list state or UI.
No public edit/cancel	Only authorized admin action may delete an internal registration; deletion is auditable, releases active capacity, and permits re-registration for the same event.
Payment separation	Keep registration status and payment status separate. For FREE events, no internal payment record or payment state is required.; paid INTERNAL events use trusted manual verification or future-

Rule	Required behavior
	provider confirmation; EXTERNAL payment is not represented internally.
WhatsApp conditionality	Show or redirect to WhatsApp only when the event has a configured link.
Confirmation email	Not mandatory. Email may remain future or optional scope.

17.1 Validation order

The recommended server-side order is: parse and schema-validate the request; load the event configuration; verify publication, eligibility, Registration Mode, form version, and registration state; route the participant through the Crescent or external path; normalize and validate a Crescent RRN only when required, using exactly 12 digits beginning with 2 and digits only; for external participants, normalize college identity by trimming leading/trailing whitespace, collapsing repeated whitespace, and applying case normalization, then normalize the external roll number; validate phone, email, custom fields, team size, leader/name rules, mixed-team policy, capacityMode, and payment configuration; check duplicates within the request; begin the database transaction or reservation workflow; enforce capacity according to PARTICIPANTS or TEAMS; check Crescent uniqueness at EVENT + normalized RRN and external uniqueness at EVENT + normalized college + normalized external roll number; for paid INTERNAL events, accept only trusted manual verification or future-provider confirmation; insert registration and responses; commit or confirm the payment-coordinated state; return a non-sensitive response. No college database or RRN verification API is consulted.

17.2 Concurrency strategy

[ARCHITECTURAL RECOMMENDATION] Use a relational database transaction with an atomic capacity reservation. For free events, the implementation may lock the event row with `SELECT ... FOR UPDATE`, calculate current active capacity inside the transaction, and insert only when capacity remains, or use an atomic conditional update. When `capacityMode = PARTICIPANTS`, the reserved unit count is the number of participating students; when `capacityMode = TEAMS`, the reserved unit count is one per submitted team. `UNLIMITED` mode applies no finite capacity restriction. For paid INTERNAL events, a temporary seat reservation may be tied to a payment attempt, with documented expiry, release on failure/expiry, and confirmation only after trusted manual verification or future-provider confirmation. External payment is never part of this internal reservation/payment state. Registration, responses, team-member inserts, and reservation/payment state must be coordinated so that concurrent attempts cannot oversubscribe the event. Active participation uniqueness is enforced for Crescent participants at EVENT + normalized RRN and for external participants at EVENT + normalized college + normalized external roll number. Authorized deletion releases the active capacity allocation; the duplicate guard must not permanently block re-registration after deletion.

For every team event, `capacityMode` is mandatory and must be either `PARTICIPANTS` or `TEAMS`. The configured value controls capacity calculations, the admin dashboard, exports, API responses, and tests. The architecture is confirmed; the selected value remains **[PENDING PER EVENT]** until CCF supplies or approves that event's operational rule. The platform must not guess.

17.3 Capacity changes and reopening

A published event may be legitimately reopened after its registration window ends, and its capacity or fields may be changed by an authorized administrator. Such a change must create and publish a new form version where the form schema is affected; historical registrations remain attached to their original immutable version.

If a capacity of 500 already has 500 active registrations and is reduced to 450, the system must not delete registrations, invalidate confirmed participants, or retroactively move them. New registration remains blocked because current active participation exceeds the reduced capacity. The admin interface must show the over-capacity condition clearly and require an explicit corrective policy decision. If capacity is increased, registration may resume when all other event conditions permit. Capacity allocations released by authorized deletion are available for new registration.

17.4 Registration Mode and External Forms

Every event has a first-class Registration Mode: INTERNAL, EXTERNAL, or NONE. INTERNAL uses the CCF website's built-in configurable form builder. EXTERNAL uses an external registration form or URL such as Google Forms; it is a legitimate supported mode, not merely an emergency fallback. For EXTERNAL, registration payment also remains fully external: the external form may include a UPI QR/payment instruction and a user-entered UPI Transaction ID / UTR, and CCF/IT may manually verify it outside the CCF internal payment database. The CCF website must not determine or pretend to know whether external payment completed. NONE means the event does not require online registration.

For EXTERNAL, the Admin Panel must provide an external registration URL, optional CTA label, and optional short instruction. The public Event Detail page displays the approved event content and a registration CTA, such as REGISTER NOW, which opens the configured URL. The platform must validate that the URL is present and valid before publication. It must not import, replicate, or treat external-form responses as internal registration records. The external form remains the source of registration data for that event. The external workflow is Event Page → External Registration URL → External Form → External Payment/Verification where applicable. The CCF internal Registration Status and Payment Status must not pretend to know the external payment result. Raw implementation details need not be exposed to visitors.

17.5 Internal CCF Event Form Builder

For INTERNAL events, IT Admins must be able to create and configure event-specific registration forms from the Admin Panel without a developer changing frontend or backend code for each new event. The builder is a focused CCF-specific product and is not intended to become a complete Google Forms clone. Published form configurations are versioned. A published form version becomes immutable for registrations that used it. Editing a published/previous form creates a new form version, existing registrations remain attached to their original version, and new registrations use the newly published version. Historical schemas and exports must remain interpretable.

System fields have platform-level meaning and may carry built-in validation or business logic. They may include participant type, college, Crescent RRN, External Roll Number, name, department, year, academic level (UG/PG), phone/WhatsApp number, email where required, team name, team membership, and an optional team leader. Supported field types include TEXT, TEXTAREA, NUMBER, EMAIL, PHONE, DATE,

TIME, DATETIME, SELECT, MULTI_SELECT, RADIO, CHECKBOX, and FILE. Each field has a defined scope: REGISTRATION, PARTICIPANT, TEAM, or TEAM_MEMBER.

Each custom field must support an admin-defined label, help text, placeholder, required/optional state, options where relevant, validation rules, minimum/maximum where relevant, and display order. Form configuration must be versioned. A submission must reference the immutable form version. Historical forms remain interpretable because their EventField definitions remain associated with the immutable FormVersion. If registration is reopened after closure, or capacity or fields are changed, the administrator publishes a new form version; v1 registrations remain attached to v1 and new registrations use v2. The exact event fields and rules are supplied by CCF per event and remain **[PENDING PER EVENT]** until provided.

17.6 Conditional Logic for Mixed Events

Eligibility determines who may participate; Registration Mode determines how they register; form configuration determines what information is collected. These are separate concerns. For mixed events where Crescent Students and Other Colleges are both enabled, the internal form should initially ask which college the participant is from and allow Crescent or Other College. The form then displays the appropriate branch without duplicating common fields.

The Crescent branch may contain Name, Crescent RRN, Department, Year, UG/PG, Phone, and other configured fields. The external branch must contain Name, College/University, External Roll Number, Department, Year, UG/PG, Phone, and other configured fields. Both branches may converge into common event fields, team configuration, and internal payment only when Registration Mode is INTERNAL. Conditional visibility and branching must be stored in the versioned form schema rather than hardcoded separately for each event.

17.7 Rich-Text Event Content

The Admin Event Editor must provide a configurable Description / Content area for event description, rules, instructions, eligibility explanations, important notes, structured lists, headings, links, and normal event content. MVP formatting should include headings, bold, italic, lists, links, and paragraphs. The rendered content appears on the public Event Detail page.

Admin-authored rich text must be sanitized before rendering. Scripts, unsafe HTML, dangerous attributes, executable content, and unsafe embeds are prohibited. Arbitrary raw HTML rendering must not be the default feature. Sanitization is required both at save time and at render time where the implementation supports defense in depth.

17.8 Payment Architecture

Some CCF events are free and some are paid. Each event therefore has a payment mode of FREE or PAID. Paid events have a configurable fee amount. Payment status is separate from registration status. RegistrationStatus values are ACTIVE and CANCELLED. PaymentStatus values are PENDING, VERIFIED, REJECTED, EXPIRED, and REFUNDED.

For free events, no internal payment record or payment state is required. For paid INTERNAL events, the confirmed MVP payment method is MANUAL_UPI, with optional future PROVIDER support. The Admin Panel may configure the UPI ID, payee name, fee amount, transaction/reference-ID requirement, and verification state. The platform will generate a dynamic UPI payment URI from trusted server-side

configuration: `upi://pay?pa=<UPI_ID>&pn=<PAYEE_NAME>&am=<AMOUNT>&cu=INR&tn=<REFERENCE>`. This URI powers both mobile UPI Intent actions and dynamically generated QR codes for desktop users. The QR code is a visual representation of the trusted URI and must not be stored as a static image asset.

The internal manual-UPI flow is: Registration → PAYMENT INITIATION (Intent or QR) → UPI APP → USER AUTHORIZATION → TRANSACTION → USER-SUBMITTED REFERENCE → PENDING PAYMENT VERIFICATION → authorized admin/IT manual verification → VERIFIED → ACTIVE. Initiation is NOT proof of payment. The server remains authoritative. PAYMENT_PROVIDER is an optional future capability and Razorpay or another gateway is not mandatory.

For EXTERNAL events, payment remains fully external to the CCF internal payment database. The external form may contain a UPI QR/payment instruction and a user-entered UPI Transaction ID / UTR, and CCF/IT may verify it manually in that external workflow. The CCF website must not determine or pretend to know the external payment result.

Paid INTERNAL events with limited capacity must not be oversubscribed when multiple participants register or pay concurrently. If the final design uses temporary reservations, it must document reservation creation, expiry, seat release, and successful manual verification. If another safe atomic strategy is selected, it must document its equivalent lifecycle. Capacity enforcement remains governed by `capacityMode` for team events and must be coordinated with internal payment state.

18 Team Registration

Teams are modeled independently from the concept of a leader. A Team belongs to an Event Registration and has zero or one optional leader reference. TeamMember rows connect participants to the Team and carry an ordinal position, participant type, and the appropriate identity fields. A team name is nullable unless required by the event. Mixed teams are allowed and may contain Crescent students, external students, or both. Participant type remains mandatory because Crescent members require Crescent RRN validation while external members require college plus external roll-number identity validation.

```
Event 1 —< Registration 1 —0..1 Team —< TeamMember
                        |
                        |— optional leader TeamMember
                        |— RegistrationResponse rows
                        |— Payment record where applicable
```

Student identity is represented only by the minimum approved fields needed for the event.

The team validator must reject fewer than the configured minimum, more than the configured maximum, missing required leader or team name, a leader that is not a team member, invalid Crescent RRN formats when a Crescent RRN is required, missing name/college/external roll number for an external participant, duplicate participant identities within the request according to their participant type, and any participant already participating in another team or registration for the same event. Mixed teams are allowed. For Crescent members, the event-scoped duplicate guard is EVENT + normalized RRN. For external members, it is EVENT + normalized college + normalized external roll number. The same identity may participate in a different separate event. The UI may offer add/remove member rows, but the server remains authoritative.

19 Recruitment System

CCF has explicitly confirmed that recruitment is currently OPEN. This is the current operating status, not a permanently hardcoded product assumption. Recruitment is manually opened and closed by administrators. Recruitment remains a separate CCF process and is not changed by the event-registration eligibility rules. Its eligibility remains Crescent students only, any academic department, and any year. External-college students may register for eligible events but may not apply for CCF recruitment merely because event participation is open to them. Each student may apply to only one CCF department. Required fields are name, RRN, department applying for, academic department, year of study, and a WhatsApp-enabled phone number. The recruitment RRN field uses the same server-side format rule: exactly 12 digits, beginning with 2, digits only after normalization. No resume, questionnaire, or interview is required. Further information is communicated through WhatsApp, and successful application should show the configured new-joiner group link.

Recruitment is manually controlled. The board or CCF instructs IT when recruitment should close. Closing removes or disables Apply Now and rejects new submissions server-side. If an administrator deletes an application, the same RRN may apply again. The admin area must support opening, closing, viewing, searching, deleting, exporting, and configuring the recruitment WhatsApp link.

20 Admin Dashboard

The dashboard is an operational console, not an unnecessarily complex CMS. Its overview should show upcoming events, active registrations, registration counts, recruitment status, recent activity, and capacity utilization by the configured PARTICIPANTS or TEAMS mode where those values are verified and useful. For the MVP, both CCF_ADMIN and IT_ADMIN have the same full operational control set.

Area	Capabilities	Protection
Dashboard	Status cards, recent activity, capacity overview	Authenticated; aggregate data only
Events	Create, edit, configure, publish, close, archive; configure eligibility, Registration Mode, form versions, form builder, rich text, team, capacity, and payment	Role and audit checks; publication validation
Payment verification	Review manual-UPI transaction/reference IDs, verify or reject INTERNAL payments, and update payment/registration states	Authenticated; authorized action and audit trail; no external payment result is inferred
Registrations	Search, filter, view, export, authorized delete; review payment and registration states	RRNs, external identifiers, and phone numbers never public
Recruitment	Open/close, search, view, export, delete, WhatsApp configuration	Authenticated; confirmation for deletion

Area	Capabilities	Protection
Members/departments	CRUD for approved public content	Publication approval and sensitive-field exclusion
Media/gallery	Upload, curate, order, attach to event, archive	Object-storage access control
Notifications	Meaningful dashboard alerts and read state	Do not notify on every registration
Settings	Site-level configuration, event-mode defaults, payment display, and operational links	Restricted; secrets never stored in content fields

Destructive actions must require confirmation and should state the scope of deletion. The interface must make event status, recruitment status, export readiness, and pending cleanup states obvious.

21 Notification System

Dashboard notifications are the primary recommendation. Useful examples include registration milestones, an event nearing capacity, registration closing soon, recruitment status changes, and important system alerts. The system must not generate one notification for every registration. Email may be added if CCF confirms an operational need. Telegram is [FUTURE SCOPE / OPTIONAL] and should not be introduced without a clear benefit.

22 Media and Event Gallery

The CCF-provided event and media images are currently stored/shared through Google Drive. Google Drive is the current source/location of supplied media only. Production media storage is finalized as Cloudflare R2. The initial supplied-media scope covers Magnora'26, FinRise'25, and FinVibe Fiesta S2. CCF should curate a reasonable number of high-quality images instead of uploading every available image. Production media storage is finalized as Cloudflare R2. The platform will store original and derived image objects in an S3-compatible bucket, record only metadata and object keys in the database, generate responsive widths and modern formats, lazy-load below-the-fold images, and create thumbnails for admin browsing. Each Media record includes event relationship, object key, alt text, width, height, file size, mime type, sort order, and publication state. Google Drive must not be treated as a public production CDN or expose private folder links.

The admin workflow is upload → validate type and size → virus or content scan where available → create optimized variants → store metadata → attach to event or gallery → preview → publish. Public media URLs should be CDN-delivered and should not reveal private bucket credentials.

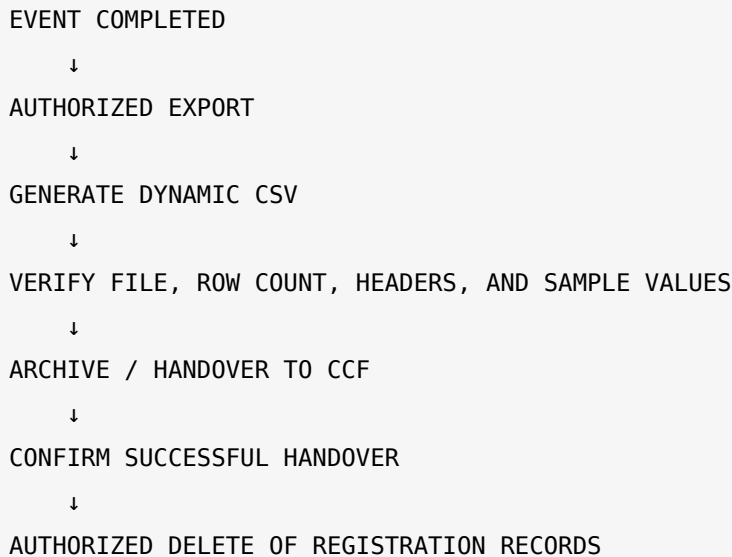
23 CSV Export, Backup, and Data Lifecycle

CSV export is critical. It must be accurate, complete, consistent, UTF-8 compatible, properly escaped, stable, and easy to open in Excel. The columns must be generated from the event's configured registration schema and the immutable form version used by each registration rather than hardcoded as one universal format.

A Crescent event may export Name, RRN, Department, Year, Phone, and Registration Date. An external-participant event may export Name, College / University name, External Roll Number, Department, Year, Phone, and Registration Date. A mixed event may export both participant paths in a stable schema with participant type. Another event may export Name, RRN, Portfolio URL, Experience, and Phone. Team exports must represent team/member relationships clearly, with stable member ordinals, participant type, normalized college where applicable, and the appropriate identifier. External payment data remains outside the CCF internal payment database; internal payment status may be exported only for INTERNAL events according to the approved event export schema.

Use the naming convention `CCF_<EVENT>_Registrations_<YYYY-MM-DD>.csv`, normalizing the event slug for filesystem safety. Use UTF-8 with BOM if Excel compatibility requires it, RFC 4180-style quoting, ISO 8601 timestamps or an explicitly approved India-local display format, and a stable column order derived from configuration order.

The required operational workflow is:



No automatic deletion is permitted without safeguards. The system should preserve a non-sensitive audit record of export, verification, handover, deletion, capacity release, re-registration, form-version changes, and payment-verification events. Deleting an authorized active/current registration releases its capacity allocation and removes it from the active duplicate guard so the participant may register again for the same event. Completed-event deletion still follows export verification, handover, and approved retention rules. Retention duration is **[PENDING CCF CONFIRMATION]**.

24 Database Architecture

The following normalized schema is an **[ARCHITECTURAL RECOMMENDATION]**. A relational database is recommended because the system benefits from transactions, unique constraints, row locking, controlled configuration, and strong indexing. The database provider is finalized as PostgreSQL, using the Prisma ORM. Sensitive fields should be minimized, access-controlled, and deleted according to the approved lifecycle. Primary keys use UUIDs, and timestamps use UTC/TIMESTAMPTZ. Asia/Kolkata is the application display timezone.

Entity	Key fields	Relationships and constraints
AdminUser	id, email, name, role, active, totp_secret, recovery_codes	Unique email; role controls dashboard actions; audit fields retained; TOTP and recovery codes for fallback/recovery
Department	id, slug, name, description, active	Unique slug and name; referenced by members and recruitment choices
Member	id, name, position, department_id, photo_media_id, bio, social_url, visibility	No RRN or private phone; public visibility explicit
Event	id, slug, name, schedule, venue, status, capacity, capacity_mode, registration_method, registration_mode, registration_opens_at, registration_closes_at, active_form_version_id, eligibility_crescent, eligibility_external, payment_mode, payment_method, fee_amount, upi_id, payee_name, external_url	Unique slug; indexed status, schedule, capacity_mode, and Registration Mode; eligibility flags are independent; payment_mode is FREE or PAID; MANUAL_UPI is the confirmed MVP internal method
EventContent	id, event_id, description_rich, rules_rich, instructions_rich, eligibility_rich, notes_rich	Sanitized rich text for public rendering
FormVersion	id, event_id, version_number, status, published_at, created_by	Published versions are immutable; registrations reference the version used; EventField definitions are associated with the version
EventField	id, event_id, form_version_id, key, label, type, system_or_custom, required, options, validation, conditional_logic, order, scope	Unique form_version_id + key; immutable versioned form schema; scope defines registration, participant, team, or member field
EventParticipant	id, event_id, registration_id, participant_type, college_normalized, identifier_normalized	Event-scoped active identity registry; enforces uniqueness; released on registration deletion

Entity	Key fields	Relationships and constraints
Registration	id, event_id, form_version_id, mode, participant_type, participant_name, college_normalized, identifier_normalized, status, payment_status, registration_code, created_at	Crescent uniqueness uses EVENT + normalized RRN; external uniqueness uses EVENT + normalized college + normalized external roll number; status is ACTIVE or CANCELLED; payment_status is PENDING, VERIFIED, REJECTED, EXPIRED, or REFUNDED
RegistrationResponse	id, registration_id, event_field_id, value	Unique registration_id + event_field_id; dynamic responses reference the immutable form version used at submission
Team	id, registration_id, name, leader_member_id	Leader nullable; one team per team registration unless later approved
TeamMember	id, team_id, participant_type, name, college_normalized, identifier_normalized, phone, academic_department, year, position	Crescent identity uses normalized RRN; external identity uses normalized college + normalized external roll number; mixed teams allowed
RecruitmentApplication	id, rrn_normalized, name, department_id, academic_department, year, phone, status	Separate Crescent-only process; one active application per RRN; deletion permits reapplication
Media	id, event_id, object_key, alt_text, width, height, mime_type, visibility, order	Object storage metadata only; Google Drive is source-media location, not assumed production backend
Notification	id, type, title, body, severity, read_at, created_at	Dashboard-only; no per-registration spam
SiteSettings	key, value, updated_by, updated_at	Allowlisted keys only; no secrets in public settings
Payment	id, registration_id, method, status, amount, currency, upi_id, payee_name, payment_uri, user_reference, verified_by, verified_at	Internal payment only; MANUAL_UPI initiation and manual verification; EXTERNAL payment has no internal payment state
AuditLog	id, actor_id, action, entity_type, entity_id, metadata, created_at	Non-sensitive operational trail

24.1 Constraints and indexes

Use UUID or equivalent non-sequential identifiers for public-facing record references, while retaining internal foreign keys. Add unique indexes for event slug, department slug, admin email, form-version field key, and active participation identity. For Crescent participants, enforce uniqueness at EVENT + normalized RRN among active/current registrations via the event_participants registry. For external participants, enforce uniqueness at EVENT + normalized college + normalized external roll number among active/current registrations; do not create a global unique college or global unique roll-number constraint. Index event status, capacity_mode, Registration Mode, eligibility flags, payment mode, payment method, and opening/closing timestamps, registration event ID and creation time, event-scoped team-member normalized identifiers, recruitment status, media event ID and order, and audit actor/time. Historical form versions and registration references must remain immutable for interpretation. Authorized deletion must release capacity and remove the deleted record from the active event_participants duplicate guard, while retaining a non-sensitive audit event.

Use foreign keys with deliberate cascade rules. Deleting an event should normally be blocked when registrations exist; archiving is preferred. Deleting an event registration is an authorized lifecycle operation: release its active capacity allocation, remove it from active uniqueness checks, retain only the approved non-sensitive audit event, and allow the participant to register again for that event. Deleting an application may be a hard delete of the minimum application record because the confirmed policy allows reapplication, but the audit log should record the actor and time without reproducing the deleted personal data. Member and media records should generally use soft visibility or archive states.

25 API Architecture

The API architecture is finalized as Next.js Route Handlers using a REST-style API with Zod validation. Server Actions may be used internally where technically useful, but they are not the primary documented API architecture. Public read endpoints should return only published and non-sensitive fields. Admin endpoints must require authenticated sessions and authorization checks. The API should return stable error codes, human-readable messages, and a request identifier for support without exposing raw database errors.

Method	Route	Auth	Behavior
GET	/api/events	Public	List published events with derived status and pagination
GET	/api/events/:slug	Public	Return published event detail, eligibility-safe content, Registration Mode, configuration-safe status, public fee/method display for INTERNAL events, and capacityMode when appropriate; never expose private external payment state or responses

Method	Route	Auth	Behavior
POST	/api/events/:slug/registrations	Public form + rate limit	For INTERNAL mode, route by eligibility, validate the active immutable form version, apply Crescent or external identity rules, honor capacityMode, coordinate internal payment where required, and create an individual/team registration transactionally. EXTERNAL and NONE modes do not use this endpoint.
GET	/api/recruitment/status	Public	Return open/closed state and safe display configuration; recruitment remains Crescent-only
POST	/api/recruitment/applications	Public form + rate limit	Validate and create one allowed application per Crescent RRN
GET	/api/departments	Public	List active departments and approved descriptions
GET	/api/members	Public	List visible member profiles without sensitive fields
GET	/api/media/:event	Public	List published, optimized media for an event
GET	/api/admin/events	CCF_ADMIN or IT_ADMIN	Search/filter events and status
POST/PATCH	/api/admin/events	CCF_ADMIN or IT_ADMIN	Create or update event content, eligibility, Registration Mode, external URL, internal form, team, capacity, and payment configuration
GET	/api/admin/registrations	CCF_ADMIN or IT_ADMIN	Search/filter registrations; sensitive output protected
POST	/api/admin/registrations/export	CCF_ADMIN or IT_ADMIN	Generate and audit dynamic CSV export
DELETE	/api/admin/registrations/:id	CCF_ADMIN or IT_ADMIN + confirmation	Delete only when policy permits; audit action
GET/PATCH	/api/admin/recruitment	CCF_ADMIN or IT_ADMIN	View or change manual recruitment settings
GET/DELETE	/api/admin/recruitment/applications	CCF_ADMIN or IT_ADMIN	Search, export, or delete applications

Method	Route	Auth	Behavior
POST/PATCH/DELETE	/api/admin/members and / departments	CCF_ADMIN or IT_ADMIN	Manage approved public content
POST/DELETE	/api/admin/media	CCF_ADMIN or IT_ADMIN	Upload metadata, attach, reorder, archive media
GET/PATCH	/api/admin/notifications	CCF_ADMIN or IT_ADMIN	Read and manage dashboard notifications
POST	/api/admin/events/:id/publish	CCF_ADMIN or IT_ADMIN	Validate required mode, URL, form, eligibility, and payment configuration before publication
POST	/api/admin/payments/:id/verify	CCF_ADMIN or IT_ADMIN	Manually verify or reject an INTERNAL MANUAL_UPI reference; update payment and registration states; audit action

25.1 Error contract

Recommended

error codes are EVENT_NOT_FOUND, REGISTRATION_NOT_OPEN, EVENT_FULL, DUPLICATE_RRN_FOR_EVENT, DUPLICATE_EXTERNAL_IDENTITY_FOR_EVENT, INVALID_RRN_FORMAT, INVALID_EXTERNAL_ROLL_NUMBER, INVALID_COLLEGE_NAME, MISSING_EXTERNAL_IDENTIFIER, INVALID_TEAM_SIZE, DUPLICATE_TEAM_MEMBER, INVALID_CAPACITY_MODE, MISSING_EXTERNAL_URL, INVALID_FIELD, PAYMENT_REQUIRED, PAYMENT_FAILED, PAYMENT_NOT_VERIFIED, UPI_REFERENCE_REQUIRED, REGISTRATION_DELETED, FORM_VERSION_NOT_FOUND, FORM_VERSION_IMMUTABLE, CAPACITY_REDUCTION_BLOCKS_NEW_REGISTRATION, RECRUITMENT_CLOSED, UNAUTHORIZED, FORBIDDEN, RATE_LIMITED, and INTERNAL_ERROR. INVALID_RRN_FORMAT means the normalized Crescent RRN is not exactly 12 digits beginning with 2; it does not mean that a college identity lookup failed. The browser should map codes to plain-language messages and never render raw stack traces or SQL errors.

26 Authentication and Authorization

The public site and student forms do not require student accounts. The admin dashboard requires authenticated sessions. The authentication architecture is finalized as Auth.js using passwordless magic links via Resend as the transactional email provider. Sign-in is restricted to an allowlist of CCF and IT email addresses. The system must not store traditional administrator passwords for the MVP.

Admin authentication requires:

- **Primary:** Auth.js Passwordless Magic Link (short-lived, single-use, server-validated tokens).
- **Fallback:** TOTP authenticator-based authentication.
- **Recovery:** One-time recovery codes for break-glass account recovery (securely hashed, never logged, regeneration invalidates previous codes).

Sessions must be secure, httpOnly, SameSite, and appropriately expired. Authentication security must not depend on Resend remaining available after a valid session exists.

Use at least two roles: CCF_ADMIN and IT_ADMIN. For the current MVP, both roles have the same full operational/admin control set. The role matrix must be enforced server-side on every admin endpoint. Export and deletion should be separately audited and require confirmation.

MVP role	Operational permissions
CCF_ADMIN	Dashboard; event creation, editing, configuration, publishing, closing, and archiving; registration search, filtering, exports, and authorized deletion; recruitment open/close, applications, export, and deletion; members; departments; media/gallery; notifications; settings; exposed backup/export workflows; and audit/activity information.
IT_ADMIN	The same full operational permissions as CCF_ADMIN for the MVP. Future refinement may restrict infrastructure-sensitive actions only when explicitly approved.

27 Security and Privacy

This is a production student-data system. Validate on the server, sanitize or safely encode output, rate-limit public forms, use secure cookies, protect secrets through environment variables, restrict database network access, apply least privilege, protect admin routes, and audit sensitive actions. External payment is an external workflow and must not be inferred or represented as a known internal payment result. For INTERNAL MANUAL_UPI events, the CCF system may store a user-submitted transaction/reference ID and authorized verification state, but it must never treat UPI Deep Link, UPI Intent, URI opening, client success, or browser return as proof of payment. CSRF protection is required when cookie-authenticated state-changing requests are used; same-site cookies and anti-CSRF tokens should be applied according to the chosen framework.

Do not expose RRN, private phone numbers, registration records, recruitment records, admin APIs, database credentials, storage credentials, or private WhatsApp configuration to public users. Avoid logging full RRNs or phone numbers. If support logs need correlation, use a short-lived request ID or a masked representation.

28 Infrastructure and Repository Architecture

The infrastructure and repository architecture are finalized. The platform is a single repository Next.js application deployed on Vercel. The stack includes Next.js, React, TypeScript, Tailwind CSS, shadcn/ui, Motion, Lucide React, PostgreSQL, Prisma, Auth.js, Resend, Cloudflare R2, and Sentry. This is the implementation baseline.

```
ccf-platform/
├─ app/                public routes and protected admin routes
├─ components/         shared UI components
├─ lib/               validation, auth, db, storage, csv, audit helpers
├─ db/                schema, migrations, seeds
├─ public/            small immutable public assets
```

└─ content/	approved editorial content and source notes
└─ tests/	unit, integration, API, accessibility, and E2E tests
└─ scripts/	export verification and operational tools
└─ docs/	handbook, runbooks, data dictionary, handover
└─ .env.example	variable names only
└─ README.md	setup and emergency procedures

Critical accounts should be organizationally owned. The domain, GitHub organization, hosting project, database, object-storage account, authentication provider, and administrative email should not depend on a single student's personal account. CCF should retain an owner or recovery role, while the IT team receives named access with least privilege. Finalized providers are Next.js, Vercel, PostgreSQL, Cloudflare R2, Resend, and Sentry.

29 Development Workflow and Team Division

Use `main` for production, `develop` for integration if the team needs it, `feature/*` for planned work, `fix/*` for defects, and `release/*` only for a release candidate. A simpler trunk-based workflow is acceptable if the team keeps pull requests, review, tests, and protected production deployment.

Commit messages should be concise and typed, such as `feat(events): add configurable event fields`, `fix(registration): prevent capacity overbooking`, and `docs(handover): update export runbook`. Every pull request should state the requirement addressed, include screenshots or test evidence when relevant, identify migration or environment-variable changes, and pass automated checks before merge.

Role	Primary responsibility	Backup responsibility
Developer 1 — platform lead	Architecture, schema, auth, deployment, code review	Registration and incident response
Developer 2 — product/UI lead	Design system, public pages, accessibility, responsive behavior	Admin UX and content tooling
Developer 3 — operations/quality lead	Events, CSV, media, testing, runbooks, data verification	Backend/API review

Development, staging, and production must be separate. Pull requests deploy to preview environments, approved changes deploy to staging, and only reviewed builds deploy to production. Rollback must use the previous known-good deployment and, when a migration is involved, a documented backward-compatible path.

30 Environment Variables and Account Controls

Never hardcode database credentials, API keys, admin secrets, email credentials, storage credentials, or private WhatsApp configuration. The repository should contain `.env.example` with names and safe placeholders only.

Variable family	Purpose
Application	Environment name, public URL, runtime mode, logging level
Database	Connection string, migration mode, pool limits
Authentication	Provider IDs, secret, session settings, admin allowlist
Storage	Bucket, endpoint, region, access key, secret key
Payment	Optional future provider credentials; MANUAL_UPI event configuration is controlled data and must not expose secrets
CSV/export	Export prefix, timezone, retention controls
Notifications	Optional email provider; no Telegram requirement in MVP
WhatsApp	Per-event or recruitment link stored as controlled configuration, never exposed in unrelated pages

31 Error Handling and Operational States

User-facing errors must be clear, safe, and actionable. The UI should preserve entered data when possible and should give a retry or contact path when the problem is not correctable by the user.

Condition	User-facing behavior
Event unavailable	Explain that the event cannot currently be found or published; provide a route back to Events.
Registration not open	Show REGISTRATION NOT OPEN until the configured opening time; do not show an active internal form.
External URL missing	Keep EXTERNAL event unpublished or show a safe configuration error; never render a broken CTA.
Registration mode none	Display event information without a registration CTA.
Registration closed	Show REGISTRATION CLOSED and the configured event details; do not show an active form.
Event full	Show EVENT FULL; do not imply a waiting list.
Duplicate registration	Explain that the participant identity has already participated in this event; for Crescent, this means the normalized RRN; for external participants, normalized college plus external roll number. Do not reveal another participant's details. A registration in a different event is not a duplicate.
External identity collision	Explain that the same external roll number is already used for the same normalized college in this event. The same roll number at a different college is allowed.

Condition	User-facing behavior
Invalid RRN	Highlight the RRN field with a precise correction message; never show this error for an external participant who does not use Crescent RRN.
Invalid team size	Explain the configured minimum and maximum without exposing internal rules not meant for students.
Duplicate team member	Identify that a member appears more than once and require correction.
Payment pending/failed	Explain that payment has not been confirmed or failed; do not treat the participant as successfully registered until the trusted state is known.
Payment not verified	Do not accept a client-only success claim; provide a safe retry/contact path.
Unsafe rich text	Reject or sanitize unsafe content before storage/rendering and show a non-technical admin message.
Server/network error	Show a retry message and support reference; log technical details privately.
Recruitment closed	Display Recruitment Closed and disable Apply Now.
Admin unauthorized	Return a safe unauthorized state and do not disclose whether a resource exists.

32 Testing Strategy

Testing must cover functional behavior, API contracts, database constraints, registration workflows, team workflows, duplicate prevention, capacity, concurrency, recruitment, admin authorization, CSV exports, media, responsive layouts, accessibility, SEO, performance, security, and cross-browser behavior.

Test group	Required cases	Pass condition
Eligibility	Crescent-only, external-only, and mixed events; UG/PG, department, and year variations	Only allowed participant types can register
Registration Mode	Internal, external, and none	Correct CTA or form behavior per mode
External identity	Same external roll number at same college, same roll number at different colleges, whitespace/case normalization, same identity across different events	EVENT + normalized college + normalized external roll number is enforced only for active/current participation
RRN	Valid Crescent RRN, invalid format, external participant without RRN	Format validation only for Crescent participants; no RRN forced for external

Test group	Required cases	Pass condition
Conditional forms	Crescent path, external path, common fields, conditional visibility	Form schema honors branching logic
Form Builder	Create, edit, delete, reorder, required, validation, custom fields, conditional logic	Admin configuration drives public form and CSV
External forms	Valid external URL, missing URL, external CTA behavior	Validated before publication; CTA opens URL
Payment	Free, paid, MANUAL_UPI initiation, missing reference, manual verify/reject, invalid client success, future-provider callback, capacity race during payment	Trusted manual verification is required for MVP; no oversubscription
Registration	Open, closed, not-yet-open, full, valid Crescent RRN, external identity, duplicate active identity within same event, valid same identity across separate events, invalid fields, server bypass	Only valid submissions commit; deleted active records may re-register
Capacity race	Capacity 100 with 101 concurrent attempts under capacityMode PARTICIPANTS mode capacityMode TEAMS	No more than the configured 100 capacity units exist
Teams	Leader required/not required, name required/not required, min/max, capacityMode PARTICIPANTS/TEAMS, duplicate event-level Crescent/external identity, valid same identity in separate event, mixed teams, participant type	Each configured rule is enforced server-side
Scale	Approximately 500 registration load, concurrent attempts, concurrent paid attempts	Platform remains stable and capacity remains accurate
CSV	Dynamic columns, mixed participant exports, team exports, custom field exports	Stable UTF-8 spreadsheet-friendly output matching form schema
Security	Rich-text sanitization, external URL validation, admin-only form modification, server-side validation, authorization	No critical security defect; no unsafe HTML rendering
Recruitment	Open, closed, duplicate RRN, delete then reapply, export	Manual state and reapplication policy work

Test group	Required cases	Pass condition
Authorization	Public access to admin/API, role mismatch, expired session	Protected data remains inaccessible
CSV	Individual fields, team fields, Unicode, commas, quotes, empty optional values	Stable UTF-8 spreadsheet-friendly output
Lifecycle	Export verification, handover, active-registration deletion, capacity release, re-registration, form-version changes, reopening, and deletion confirmation	No unsafe deletion; deleted active records release capacity and may re-register
Accessibility	Keyboard-only, screen reader labels, focus, contrast, reduced motion	No critical blocker; errors understandable
Performance	Mobile throttling, image-heavy pages, event queries	Responsive and acceptable under agreed targets
Security	Rate limit, injection, CSRF where applicable, secret exposure, logs	No critical security defect
Authentication	Magic link request/expiry/reuse, allowlist rejection, TOTP fallback, recovery code single-use/regeneration, session security	Secure, single-use, and auditable auth
Dynamic QR	Correct UPI URI generation, trusted amount/payee/ID, QR representation, client amount-tamper rejection	Correct initiation without faking success
R2 Storage	Upload, validation, metadata, optimization, public delivery, invalid file rejection, credential isolation	Safe and optimized media handling
Database/Prisma	Constraints, foreign keys, unique participation registry, transaction rollback, concurrent registration	Data integrity and concurrency safety

33 Acceptance Criteria

The MVP is accepted only when public pages contain no sensitive student data; admin access is authenticated via Auth.js magic links; TOTP fallback and recovery codes work securely; both admin roles have full MVP operational permissions; events are configurable without code changes; Crescent-only, external-only, and mixed events work; internal registration, Google Form/external registration, and NONE mode work; Crescent RRN validation and external identity uniqueness work; duplicate active participation in the same event is rejected; team registration, mixed teams, and configurable team rules work; capacity mode is enforced correctly and concurrent registration cannot overbook; authorized deletion releases capacity and identity locks; dynamic forms work with immutable versions; manual UPI works with dynamic QR generation and UTR verification; external payment remains external; dynamic

CSV, Cloudflare R2 media, and recruitment manual controls work; dashboard notifications do not spam; security, accessibility, and responsive behavior pass; and handover documentation is complete.

Additional acceptance evidence must include a successful staging export, an export verification record, a tested recovery path, a role-permission matrix, an environment-variable checklist, and a handover sign-off by CCF and the IT lead.

34 MVP and Future Scope

The launch MVP is the smallest complete platform that supports public identity, verified informational pages, events, individual/team registration, recruitment, optional WhatsApp actions, an authenticated admin console, CSV exports, media/gallery, security, SEO, accessibility, performance, and responsive behavior.

Certificate generation, attendance, automated certificates, email automation, Telegram notifications, advanced analytics, QR check-in, student profiles, event reminders, alumni systems, newsletters, sponsors, advanced CMS features, and expanded analytics are **[FUTURE SCOPE]**. Payment configuration and payment-state architecture are MVP requirements, while the provider and payment/refund policy remain pending. Future features should be represented in the architecture only through clean extension points and must not complicate launch unnecessarily.

35 Requirements Traceability Matrix

Requirement	Feature / component	API / data	Test evidence
Participant identity per event	Registration guard and identity normalizer	Crescent EVENT + RRN; external EVENT + normalized college + external roll number	Same-college external duplicate fails; different-college same roll number succeeds; separate event succeeds
Configurable capacity mode	Status and atomic counter	EventStatus + capacityMode admin control; transactional registration	PARTICIPANTS/TEAMS capacity race tests
Event eligibility	Eligibility guard and registration routing	Event eligibility flags	Crescent/external/mixed tests
Registration Mode & Method	Mode/Method-based CTA and registration form	Event registration_mode + registration_method	Internal/external/none and built-in/google-form tests
Registration Window	Time-based registration gate	registration_opens_at + registration_closes_at	Pre-open and post-close rejection tests
Form Builder	Dynamic form configuration	FormVersion + EventField schema	Field creation, reorder, required, validation, and conditional-logic tests

Requirement	Feature / component	API / data	Test evidence
Payment separation	External workflow and internal MANUAL_UPI flow	No external internal payment state; Payment entity for INTERNAL	External payment remains external; manual-UPI initiation and verification tests
Rich-text content	Sanitized content editor and renderer	EventContent rich fields	Sanitization and rendering tests
Form versioning	Immutable historical schema	FormVersion + versioned registration	Reopen/edit creates v2; v1 registrations remain interpretable
Capacity changes	Safe reopen and reconfiguration	Event capacity + audit	Reduction does not invalidate existing registrations; increase may resume registration
Configurable team rules	Team builder and validation summary	Participant type, team config, capacityMode	Leader/name/size, mixed-team, and identity matrix
Optional WhatsApp	Success action	Event/recruitment settings	Link shown only when configured
Dynamic CSV	Schema-driven export workflow	Export endpoint + immutable form schema	Unicode, external, mixed, team, custom-field, and empty-field tests
Safe deletion and re-registration	Lifecycle gate and capacity release	AuditLog + active-identity constraints	Authorized deletion releases capacity and permits same-event re-registration
No public sensitive data	Privacy boundary and public DTOs	Protected admin queries	Public/admin access test
Manual recruitment closure	Join Us state	RecruitmentStatus + form	Closed submission test; current status stated as open
Final technology stack	Next.js, Prisma, PostgreSQL, Auth.js, Cloudflare R2, Resend	Application configuration	Baseline setup and test pass
Admin Authentication	Auth.js, Magic Link, TOTP, Recovery codes	Auth.js provider + database	Auth flow, fallback, and recovery tests
Identity registry	event_participants uniqueness registry	EventParticipant entity + constraints	Duplicate active identity rejection tests
Dynamic UPI QR	Dynamic QR generation from trusted URI	UPI initiation logic	Correct QR/URI representation and amount tests

Requirement	Feature / component	API / data	Test evidence
Cloudflare R2	S3-compatible object storage	Media entity + R2 client	Upload, optimization, and delivery tests
Future handover	Runbooks and documentation	Repository, deployment, and schema docs	Fresh-environment setup and supervised maintenance test

36 Development Roadmap

36.1 Phase 0 — Requirements and Architecture

Confirm official facts, resolve conflicts, approve pending decisions, freeze MVP, and approve this handbook. The definition of done is signed-off requirements and an approved privacy boundary.

36.2 Phase 1 — Repository and Project Setup

Setup Next.js, TypeScript, Tailwind, shadcn/ui, and baseline configuration.

36.3 Phase 2 — Database Setup

Prisma + PostgreSQL setup, migrations, and seed structure.

36.4 Phase 3 — Authentication

Auth.js + Resend Magic Link + TOTP + recovery codes implementation.

36.5 Phase 4 — Design System and Layout

Implement tokens, typography, navigation, footer, and responsive shells.

36.6 Phase 5 — Public Website

Build Home, About, Departments, Members, Contact, Join Us status, and SEO metadata.

36.7 Phase 6 — Event Management

Event architecture and admin event management.

36.8 Phase 7 — Dynamic Form Engine

Implement Form Builder, immutable versions, and conditional logic.

36.9 Phase 8 — Registration Engine

Implement registration logic, event_participants registry, and race-safe capacity.

36.10 Phase 9 — Team Registration

Implement team, member, mixed-team support, and validation.

36.11 Phase 10 — Payment

Manual UPI + dynamic UPI URI + QR + UTR verification.

36.12 Phase 11 — Recruitment

Implement manual open/close state, application form, and admin controls.

36.13 Phase 12 — Admin Dashboard

Operational tables, search, filters, role checks, notifications, and audit logs.

36.14 Phase 13 — CSV and Media

Dynamic schema-driven CSV and Cloudflare R2 media integration.

36.15 Phase 14 — Security and Testing

Run full functional, security, accessibility, SEO, performance, and concurrency tests.

36.16 Phase 15 — Deployment

Vercel deployment, Sentry monitoring, logs, and backup/recovery.

36.17 Phase 16 — Handover

Deliver repository access, credentials, runbooks, and future-change guide.

37 Deployment, Backup, and Recovery

Production deployment must use protected branches and review. Database migrations must be backward-compatible where possible and must be tested on staging. Backups should be managed by the database provider plus an operational export procedure for event registrations. Media storage should have versioning or a recovery strategy where supported. The team must document who can restore the database, who can access storage, how to rotate secrets, and how to roll back the application.

A recovery drill should be performed before launch and periodically thereafter. The drill should restore a non-production copy, verify schema and media references, check that public pages remain free of sensitive data, and record the time and responsible operator. The exact recovery frequency is [PENDING CCF CONFIRMATION].

38 Antigravity Implementation Brief

The implementation agent should treat this handbook as the source of truth and should not begin by inventing content. The first implementation task is to create the approved repository structure, environments, database migration plan, and content-validation checklist. Any missing CCF information must become a visible pending item rather than a guessed value.

The implementation technology stack is finalized. The platform will be built as a Next.js full-stack application using TypeScript, React, Tailwind CSS, shadcn/ui, and Motion. The backend uses Next.js Route Handlers with Zod validation. Data is managed in a PostgreSQL database using the Prisma ORM. Admin authentication is provided by Auth.js using passwordless magic links via Resend, with TOTP fallback and one-time recovery codes. Media storage is finalized as Cloudflare R2. Deployment is on Vercel, with Sentry for monitoring. This stack is the implementation baseline and must not be changed without justification. Antigravity must not invent CCF content, members, or leadership.

Subsystem	Implementation requirement
Public CCF website	Home, About, Departments, Members, Contact, Join Us status, and Event Detail pages.
Event management	Admin CRUD for events, content, eligibility, mode, fields, teams, and payment.
Dynamic event content	Sanitized rich-text description, rules, instructions, and notes.
Event eligibility	Independent Crescent and Other Colleges flags; mixed-event support.
Internal Form Builder	Admin configuration of system and custom fields with validation; not a full Google Forms clone.
Conditional logic	Branching Crescent/external paths and conditional field visibility in internal forms.
External registration	Google Form or external URL support with validated CTA behavior; external payment/verification remains outside CCF internal payment state.
Team registration	Mixed teams explicitly allowed; participant type retained; Crescent RRN and external college+roll identity rules; configurable leader, name, size, and capacity mode.
Capacity management	Atomic PARTICIPANTS or TEAMS enforcement; race-safe internal reservation; authorized deletion releases capacity; safe capacity reduction/increase behavior.
Payment integration	INTERNAL FREE/PAID configuration; MANUAL_UPI MVP initiation and manual verification; PAYMENT_PROVIDER optional future capability; EXTERNAL payment stays external.
Registration management	Admin search, filter, view, export, and authorized deletion.
Dynamic CSV export	Columns generated from event-specific form schema; spreadsheet-friendly.
Recruitment	Crescent-only application process; separate from event eligibility.
Media/gallery	Google Drive source-media curation; pending production object storage.
Admin dashboard	Full operational access for both CCF_ADMIN and IT_ADMIN for MVP.
Notifications	Meaningful dashboard alerts; no per-registration spam.
Audit logging	Non-sensitive trail of administrative and sensitive operational actions.
Security	Server-side validation, sanitization, authorization, and student-data protection.
Data lifecycle	Export, verify, archive, and safeguarded deletion.

Subsystem	Implementation requirement
Handover	Organizational account ownership, runbooks, and future-batch readiness.

Implementation area	Required handoff content
Routes	Exact public and admin route map in Section 9; no unapproved routes that expose sensitive data.
Schema	Entities, relations, indexes, unique constraints, lifecycle, and deletion rules in Section 24.
API	Endpoint categories, auth requirements, request validation, stable error codes, and response boundaries in Section 25.
Authentication	Allowlisted admin identity, role matrix, session/cookie policy, recovery owner in Section 26.
UI architecture	Reusable navigation, event, registration, team, form, status, gallery, admin-table, modal, and notification components.
Environment	Separate development, staging, production variables; no secrets in Git.
Testing	Test matrix, concurrency cases, export cases, accessibility, security, and cross-browser evidence.
Deployment	Protected production branch, staged migrations, preview review, rollback, backup, and recovery.
Handover	Organizational account ownership, runbooks, data dictionary, decision register, and supervised maintenance task.

39 Handover Checklist

Before handover, the IT team must provide the repository URL, protected-branch policy, deployment project ownership, domain ownership, database access ownership, storage ownership, authentication owner, environment-variable inventory, schema and migration guide, API documentation, event-configuration guide, registration/export guide, data-deletion workflow, backup/recovery runbook, monitoring and incident process, accessibility and security checklist, and list of open pending decisions. CCF must confirm the public content, leadership names, faculty/coordinator information, event fields, recruitment policy, capacity semantics, data retention, domain, and operational account ownership. Handover is incomplete if a future batch cannot deploy a safe staging build or complete a verified CSV export.

40 Final Quality Audit

The handbook has been structured to preserve official CCF terminology, distinguish facts from recommendations, and avoid invented data. Version 0.5 finalizes the technology stack (Next.js, Prisma, PostgreSQL, Auth.js, Cloudflare R2, Resend), admin authentication (Magic Link, TOTP, Recovery

codes), and the `event_participants` identity registry. The system supports individual and mixed team registration, 12-digit Crescent RRN format validation, external identity uniqueness (EVENT + normalized college + roll number), and race-safe capacity modes (PARTICIPANTS, TEAMS, UNLIMITED). Registration is independently configurable per event, including eligibility, registration mode/method, internal form building, conditional logic, immutable form versions, and manual UPI initiation with dynamic QR generation. Recruitment is currently open and manually controllable. The handbook protects student data, requires full MVP admin access for both CCF_ADMIN and IT_ADMIN, and provides a 16-phase implementation roadmap. The leadership board is confirmed as Remi Kayalvizhi — President, Fizza Fathima — Vice President, and Zayan Ahmed — Managing Director; the official prepared-by field is Rohith Y - IT Team.

The remaining approval gate is not technical completeness but confirmation of the pending CCF decisions. No implementation should publish unconfirmed names, member hierarchy, faculty information, event registration fields, team-capacity meaning, external refund/payment policy, or data-retention policy. External identity fields and active uniqueness semantics are confirmed; exact per-event field choices and values remain event-specific.

v0.5 Change Log

This revision preserves the valid Version 0.4 handbook and incorporates the following confirmed decisions from the v0.5 revision prompt and final correction pass.

Decision	v0.5 treatment
Final technology stack	Confirmed Next.js, React, TypeScript, Tailwind CSS, shadcn/ui, Motion, PostgreSQL, Prisma, Auth.js, Resend, Cloudflare R2, and Sentry.
Admin Authentication	Auth.js Passwordless Magic Link (primary), TOTP (fallback), and one-time recovery codes (break-glass).
Identity registry	New <code>event_participants</code> entity to enforce event-scoped active participant uniqueness for both Crescent and external students.
Dynamic Form Engine	Finalized field types and scope; immutable published versions; conditional logic support.
Manual UPI + QR	Dynamic UPI URI and QR generation from trusted server-side configuration; manual UTR verification.
Capacity modes	Finalized PARTICIPANTS, TEAMS, and UNLIMITED modes with concurrency-safe atomic reservations.
Media storage	Finalized Cloudflare R2 as production object storage; Google Drive as source-media location.
Recruitment status	Confirmed as currently OPEN; manual control remains supported.

Decision	v0.5 treatment
Implementation Roadmap	Updated to a 16-phase implementation sequence reflecting the finalized stack.
Antigravity Brief	Revised to emphasize the handbook as the source of truth and the finalized implementation baseline.

v0.5 Final Correction Pass

This correction pass standardized the document for implementation readiness.

Decision	Correction treatment
Infrastructure Contradictions	Resolved all pending-state contradictions; confirmed Cloudflare R2 and finalized stack.
Capacity Terminology	Standardized to PARTICIPANTS, TEAMS, and UNLIMITED globally.
Registration Logic	Restored <code>RegistrationMethod</code> and added <code>registration_opens_at</code> / <code>registration_closes_at</code> configuration fields.
Database Enums	Standardized <code>RegistrationStatus</code> (ACTIVE, CANCELLED) and <code>PaymentStatus</code> (PENDING, VERIFIED, REJECTED, EXPIRED, REFUNDED).
Schema Cleanup	Removed obsolete <code>seats_used</code> and redundant <code>schema_snapshot</code> fields.
Form Versioning	Clarified immutable versioned definition architecture.
Auth Security	Clarified secure storage for TOTP secrets and hashed one-time recovery codes.
API Architecture	Froze Next.js Route Handlers and REST-style API as the primary documented architecture.
Concurrency Safety	Clarified mandatory invariant: concurrent users must not overbook finite-capacity events.

41 References and Source Materials

The handbook is based on the materials supplied in this conversation and does not rely on invented external facts.

No.	Source	Use in this handbook
1	CCF master prompt, “MASTER PROMPT — CRESCENT CLUB OF FINANCE WEBSITE,” supplied attachment	Primary product, architecture, security, UX, data, testing, MVP, roadmap, and handover requirements.
2	CLUBEVENTS.pdf, supplied attachment	Official overview, vision, mission, objectives, leadership structure, event

No.	Source	Use in this handbook
		history, admin board, and coordinator names.
3	ccf_logo_edited.webp, supplied attachment	Primary visual source of truth for the recommended CCF design direction.
4	Supplied website screenshots and member/event poster references	Navigation, visual direction, existing website patterns, and supplemental asset review.
5	Official Instagram: https://www.instagram.com/crescentcluboffinance/	Official social link supplied by CCF; publish only as confirmed.
6	Official LinkedIn: https://www.linkedin.com/company/ccf-2024/	Official social link supplied by CCF; publish only as confirmed.
7	pasted_content_3.txt, supplied update prompt	Confirmed Version 0.2 corrections: RRN format scope, event-level uniqueness, capacityMode, admin permissions, Google Drive source-media status, confirmed leadership, and prepared-by field.
8	pasted_content_4.txt, supplied update prompt	Confirmed Version 0.3 corrections: configurable eligibility, Registration Mode, internal form builder, conditional logic, rich-text content, payment architecture, and 500-student planning target.
9	pasted_content_5.txt, supplied update prompt	Confirmed Version 0.4 corrections: external identity composition, external-payment separation, manual UPI, deletion/re-registration, immutable form versions, capacity changes, mixed teams, recruitment status, numbering, traceability layout, and change-log requirement.
10	pasted_content_6.txt, supplied update prompt	Confirmed Version 0.5 corrections: finalized technology stack, admin authentication (Magic Link, TOTP, Recovery), identity registry, dynamic form engine, manual UPI with QR generation, Cloudflare R2 media, and 16-phase roadmap.
11	pasted_content_7.txt, final correction prompt	Confirmed v0.5 Final corrections: stack/media freeze, capacity standardization, registration method/window, enum

No.	Source	Use in this handbook
		standardization, schema cleanup, and auth security.

End of handbook. Version 0.5 is the finalized requirements and architecture source of truth. It has been frozen for implementation. Remaining PENDING items are genuine CCF operational or content decisions that should be resolved where they affect launch.